

DEEPPFAKE SHIELD

Multi-Modal GenAI Fraud Detection Pipeline

Project Requirement Document (PRD)

Version	1.0
Date	September 20, 2026
Status	Complete
Document Type	Project Requirement Document

TABLE OF CONTENTS

1. Executive Summary
2. Problem Statement
3. Solution Overview
4. Project Objectives
5. Scope & Features
6. Technical Architecture
7. Technology Stack
8. Database Design
9. System Components
10. User Stories & Use Cases
11. Acceptance Criteria
12. Risk Analysis
13. Testing Strategy
14. Deployment Plan
15. Constraints & Limitations
16. Future Roadmap

1. EXECUTIVE SUMMARY

DeepFake Shield is a sophisticated Flask-based web application designed to detect synthetic and manipulated digital content across multiple modalities including images, text, audio, and video. The system leverages state-of-the-art deep learning models to provide users with AI-powered risk assessments while maintaining honest uncertainty handling through an inconclusive classification category.

Key Highlights: Multi-modal detection (4 modalities), User authentication & history tracking, Real-time analysis with web interface, Honest uncertainty handling, Open-source architecture.

2. PROBLEM STATEMENT

Challenge: The rapid advancement of generative AI technologies has made it increasingly difficult for users to identify synthetic or manipulated digital content. Deepfakes and AI-generated misinformation pose significant risks to individuals and organizations in terms of identity fraud, reputation damage, and spread of false information.

Current Gaps: Existing detection tools are fragmented across different platforms, requiring users to switch between multiple applications. Most solutions lack transparent uncertainty handling and make overconfident predictions on ambiguous content.

3. SOLUTION OVERVIEW

DeepFake Shield provides a unified, accessible platform for analyzing digital content. The system integrates specialized deep learning models for each content modality, employs threshold-based decision logic with explicit inconclusive categorization, and maintains a comprehensive audit trail of all analyses.

4. PROJECT OBJECTIVES

- Provide accurate detection of AI-generated and manipulated content across multiple modalities
- Deliver an intuitive, user-friendly web interface accessible to non-technical users
- Maintain transparent uncertainty handling through honest inconclusive results
- Build a secure, scalable system with user authentication and private analysis history
- Enable proper evaluation and threshold tuning using labeled datasets
- Document limitations clearly and avoid overconfident claims

5. SCOPE & FEATURES

In Scope:

- Image Detection: CNN-based deepfake detection for JPEG, PNG, GIF formats
- Text Analysis: RoBERTa-based AI-generated text detection (English, 80+ words)
- Audio Detection: Synthetic voice detection (speech, up to 10 minutes)
- Video Analysis: Frame sampling + audio analysis with conflict detection
- User Authentication: Secure login/signup with hashed passwords
- Analysis History: Persistent storage and filtering of past analyses
- Dashboard: Real-time statistics and analytics
- Reports: CSV export and detailed analysis records

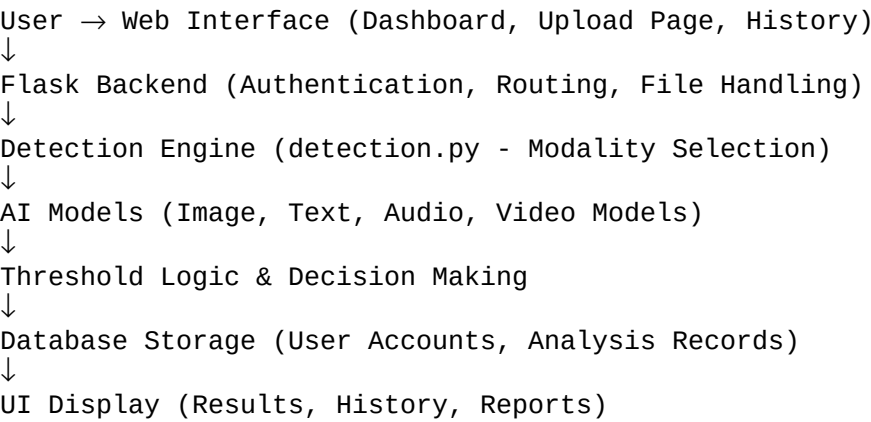
Out of Scope:

- Fact-checking or content verification
- Lip-sync analysis for video
- Music or non-speech audio detection
- Non-English language support in current phase
- Real-time stream processing

6. TECHNICAL ARCHITECTURE

The system follows a layered architecture with three primary components: Frontend (HTML/CSS/JavaScript), Backend (Flask), and AI/ML Layer (Detection Engine).

Architecture Flow:



7. TECHNOLOGY STACK

Component	Technology	Purpose
Backend Framework	Flask 2.x	Web application server
Database	SQLite/PostgreSQL	User & analysis storage
ML Framework	PyTorch + Transformers	Deep learning inference
Frontend	HTML5, CSS3, JavaScript	User interface
Authentication	Flask-Login, Werkzeug	Session management
Image Processing	PIL/Pillow	Image preprocessing
Audio Processing	librosa, soundfile	Audio analysis
Video Processing	OpenCV	Frame extraction
ORM	SQLAlchemy	Database abstraction
Testing	Pytest	Unit & integration tests
Deployment	Python 3.11+	Runtime environment

8. DATABASE DESIGN

Users Table:

Stores user account information with columns: id (PK), name, email (unique), password_hash, created_at. Passwords are hashed using werkzeug.security for security.

Analysis Table:

Records all content analyses with columns: id (PK), user_id (FK), content_type (image/text/audio/video), source_name, image_path (optional), label (result), confidence (model score), is_threat (boolean), warnings (JSON), created_at. Enables comprehensive history tracking and reporting.

9. SYSTEM COMPONENTS

app.py (Backend Controller): Handles Flask application initialization, user authentication, routing, file uploads, and database operations. Coordinates between frontend requests and detection engine.

detection.py (AI Engine): Core detection logic with specialized modules for each modality. Manages model loading, preprocessing, inference, and threshold-based decision making.

HTML Templates: 8 main pages: Home (landing), Login/Signup (authentication), Dashboard (overview), Analyze (detection interface), History (past analyses), Reports (analytics), Settings (account management).

Database Layer: SQLAlchemy ORM for database abstraction. Supports SQLite (development) and PostgreSQL (production).

Models: Pre-trained transformers from Hugging Face: dima806 (image), Hello-SimpleAI (text), MelodyMachine (audio).

10. USER STORIES & USE CASES

Journalist Verification: A journalist receives a video and needs to verify if it's authentic before publishing. Uses video analysis to detect deepfakes and gets detailed frame-by-frame evidence.

AI Safety Researcher: Researcher tests new deepfake generators against the detector. Uses `evaluation_model.py` to measure precision/recall on labeled datasets.

Social Media Moderator: Reviews flagged content for authenticity. Uses the dashboard to batch-analyze suspicious images and texts, marking inconclusive results for manual review.

Organization Protection: Company implements DeepFake Shield to protect employee identities. Sets up threshold tuning and monitors detection performance over time.

Content Creator: Creator wants to verify they haven't uploaded manipulated content. Quickly analyzes images before posting to social media.

11. ACCEPTANCE CRITERIA

1. All 4 modalities (image, text, audio, video) successfully detect test samples
2. Smoke tests pass with all models loading and running inference
3. User can register, login, upload content, and view results without errors
4. Analysis history persists and can be filtered by content type and result
5. Inconclusive results are properly marked and counted separately
6. System handles edge cases: low resolution, unsupported languages, silent audio
7. CSV export includes all analysis metadata and results
8. Performance: image analysis <2 sec, text <3 sec, audio <5 sec, video <10 sec
9. No accuracy claims without proper evaluation on labeled test data

12. RISK ANALYSIS

Risk	Impact	Probability	Mitigation
Model Accuracy Drift	High	Medium	Regular evaluation on new datasets
False Positives	High	Medium	Conservative thresholds, inconclusive category
Data Privacy	High	Low	Local inference, encrypted storage
Adversarial Attacks	Medium	Medium	Future: adversarial evaluation
Performance Bottleneck	Medium	Low	GPU optimization, batch processing
Model Availability	Medium	Low	Cache weights locally, fallback

13. TESTING STRATEGY

Unit Tests (pytest):

34 tests covering label validation, abstention logic, sampling, audio segmentation, video frame extraction, and threshold binding. All pass ■

Integration Tests:

Smoke tests verify all 4 models load and run basic inference on generated fixtures. Smoke tests are NOT accuracy benchmarks.

Evaluation Strategy:

Proper accuracy measurement requires `evaluate_model.py` with labeled test datasets. Metrics: precision, recall, F1, ROC-AUC, Brier score. Thresholds tuned on validation set only.

14. DEPLOYMENT PLAN

Development: Python 3.11+, local SQLite, Flask development server

Testing: pytest suite, smoke tests, manual QA

Staging: Docker containerization, PostgreSQL, environment variables

Production: Gunicorn/uWSGI, nginx reverse proxy, PostgreSQL, SSL/TLS, rate limiting

Monitoring: Logs, performance metrics, model accuracy tracking

Backup: Daily database backups, model weight versioning

15. CONSTRAINTS & LIMITATIONS

Model Limitations:

- Image model: Cannot detect novel generators, concept drift from training data
- Text model: English-only, trained on human/ChatGPT (v1), no fact-checking
- Audio model: Speech-only, not validated for music or general audio
- Video: 32-frame sampling cannot detect brief edits, no lip-sync analysis

System Constraints:

• Max text: 500K characters • Max audio: 10 minutes • Max video: practical limits on duration • No guaranteed accuracy without labeled evaluation data • Scores are uncalibrated model outputs

16. FUTURE ROADMAP

Phase 2 (Q2 2024): API endpoints, batch processing, webhook notifications, model ensemble methods

Phase 3 (Q3 2024): Real-time stream processing, adversarial evaluation, threshold auto-tuning

Phase 4 (Q4 2024): Multi-language support, mobile app, advanced analytics dashboard

Phase 5 (2025): Federated learning for continuous improvement, integration with social platforms